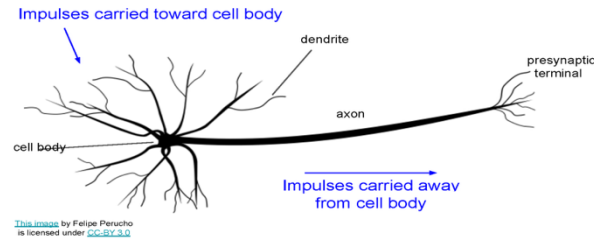


Multi-layer Perceptron

Instructor: Hyunwoo J. Kim



Annoucement

- Coding Assignment 4 has released (due on 5/26)
 - Prompt Optimization for 6-way question classification

Today's agenda

123

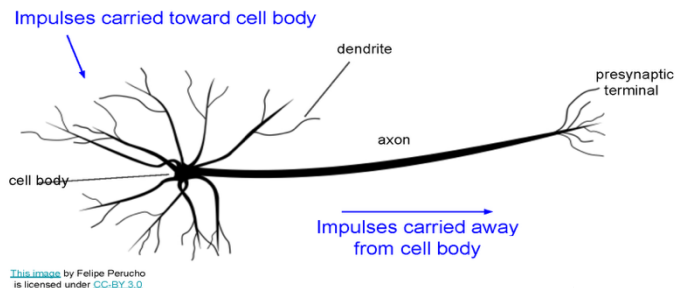
- Introduction to Neural Networks
 - Motivation
 - Feedforward networks
 - Activation functions
 - Demo

Neural networks

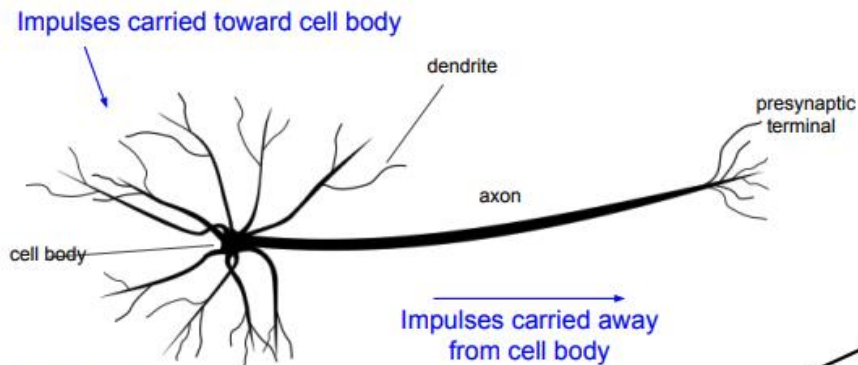
- Very coarse generalization:
 - Linear functions chained together and separated by non-linearities (*activation functions*), e.g. “max”

$$f = W_3 \max(0, W_2 \max(0, W_1 x))$$

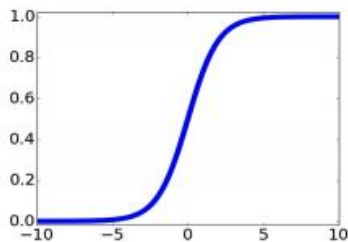
- Why separate linear functions with non-linear functions?
- *Very roughly* inspired by real neurons



Perceptron

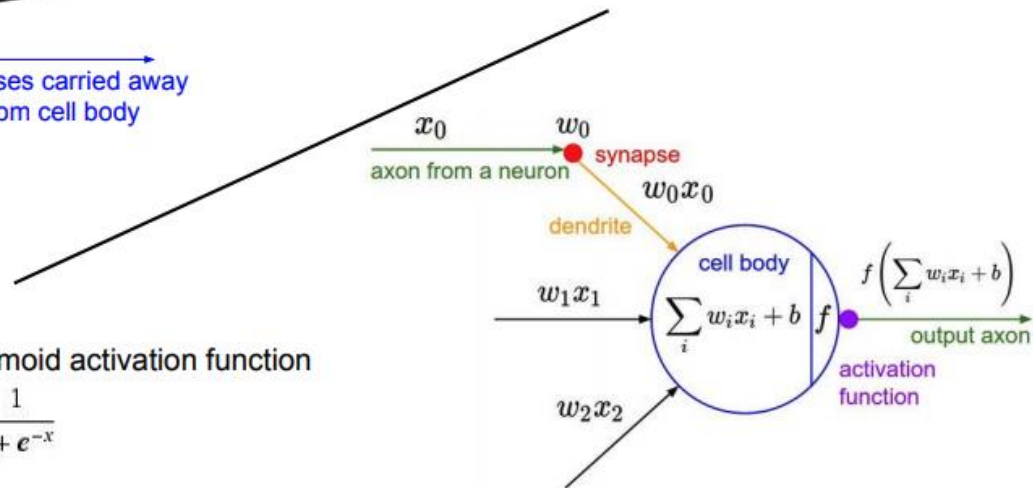


This image by Felipe Peruchio
is licensed under [CC-BY 3.0](#)



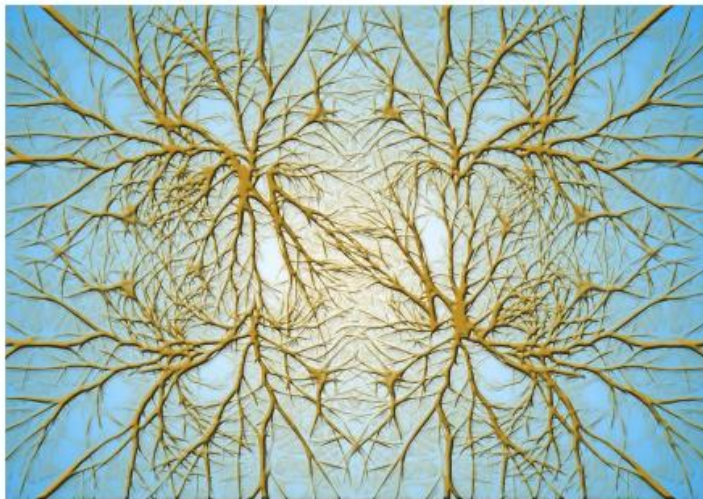
sigmoid activation function

$$\frac{1}{1 + e^{-x}}$$



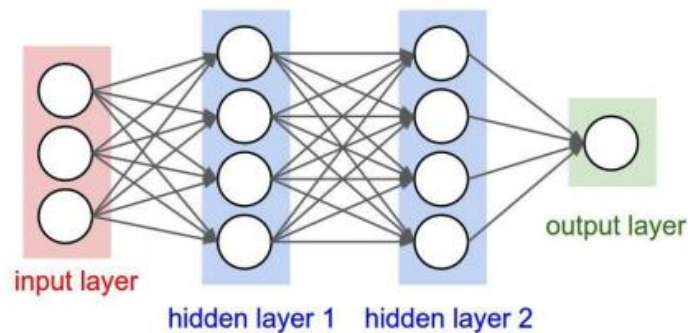
Complex connectivity patterns

Biological Neurons:
Complex connectivity patterns



[This image is CC0 Public Domain](#)

Neurons in a neural network:
Organized into regular layers for
computational efficiency



$$f(x) = \text{softmax}(W_3 \max(0, W_2 \max(0, W_1 x)))$$

Neural networks

(**Before**) Linear score function: $f = Wx$

Neural networks

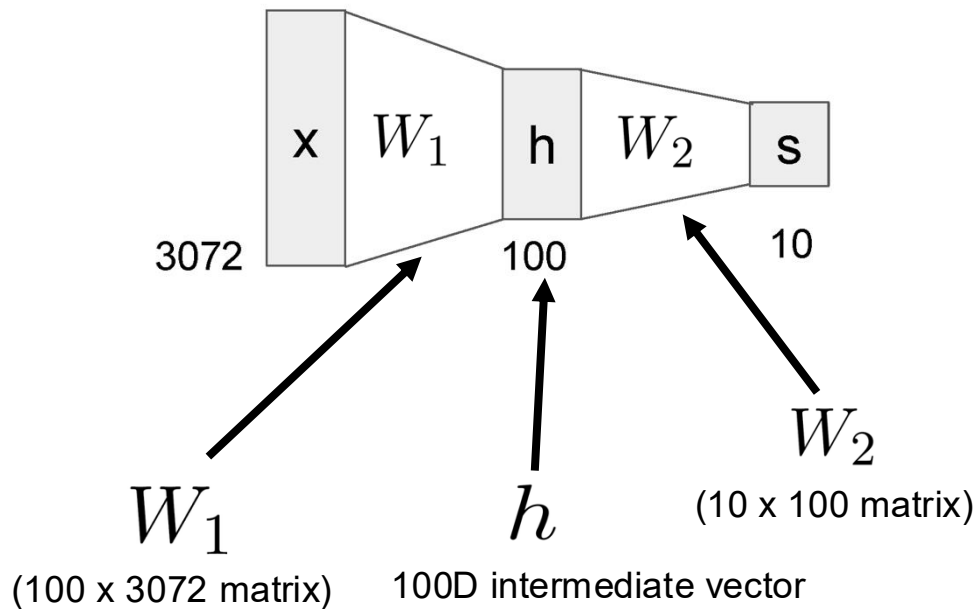
(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

Neural networks

(**Before**) Linear score function: $f = Wx$

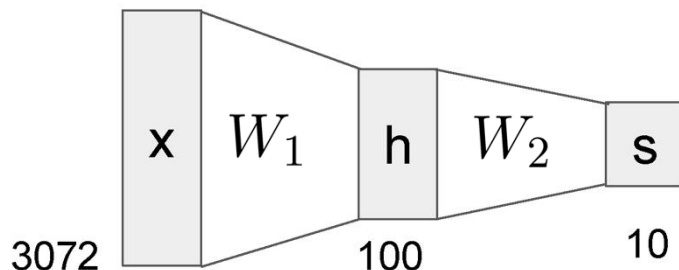
(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$



Neural networks

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

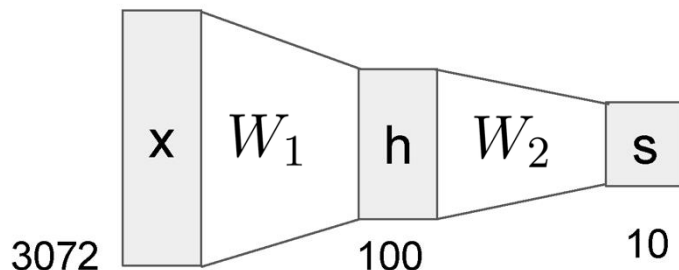


- Total number of weights to learn:
 $3,072 \times 100 + 100 \times 10 = 308,200$

Neural networks

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$



- Total number of weights to learn:
 $3,072 \times 100 + 100 \times 10 = 308,200$

Lots of parameters!
Rote learning,
overfitting to small data

Neural networks

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$
or 3-layer Neural Network

$$f = W_3 \max(0, W_2 \max(0, W_1 x))$$

Name of activation function?

Neural networks

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$
or 3-layer Neural Network

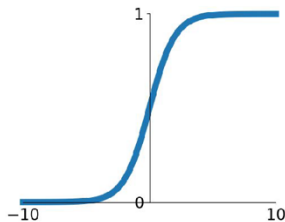
$$f = W_3 \max(0, W_2 \max(0, W_1 x))$$

What if the activation function is removed?

Activation functions

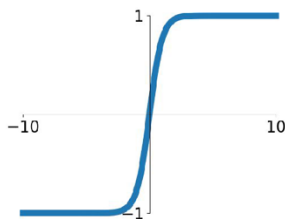
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



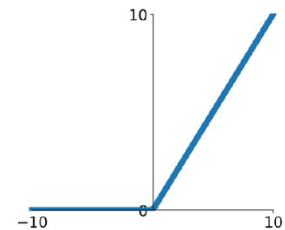
tanh

$$\tanh(x)$$



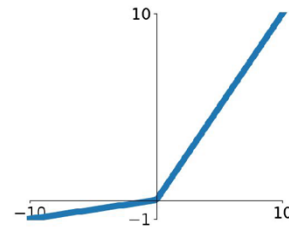
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

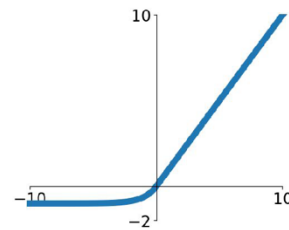


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

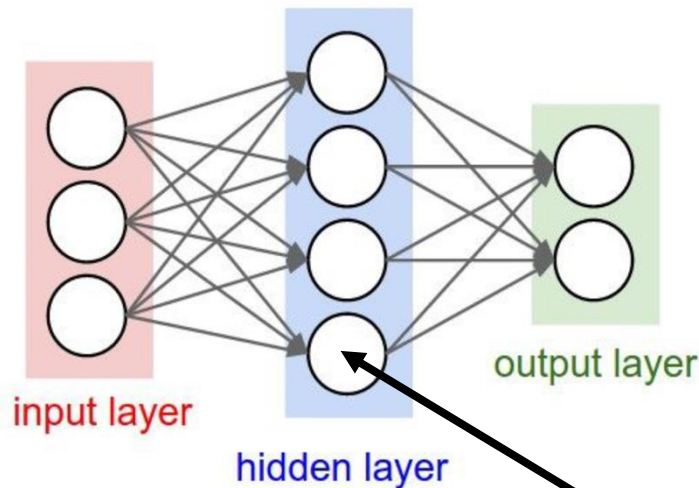
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Neural network architecture

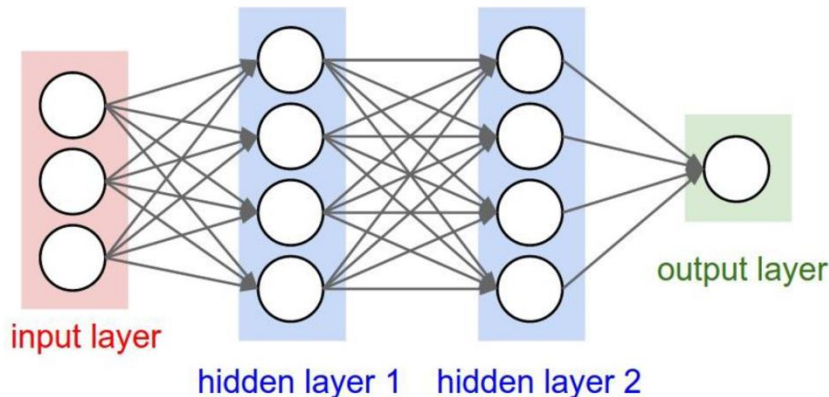
- Computation graph for a 2-layer neural network



$$f(x) = \text{softmax}(W_2 \max(0, W_1 x))$$

Neuron or unit

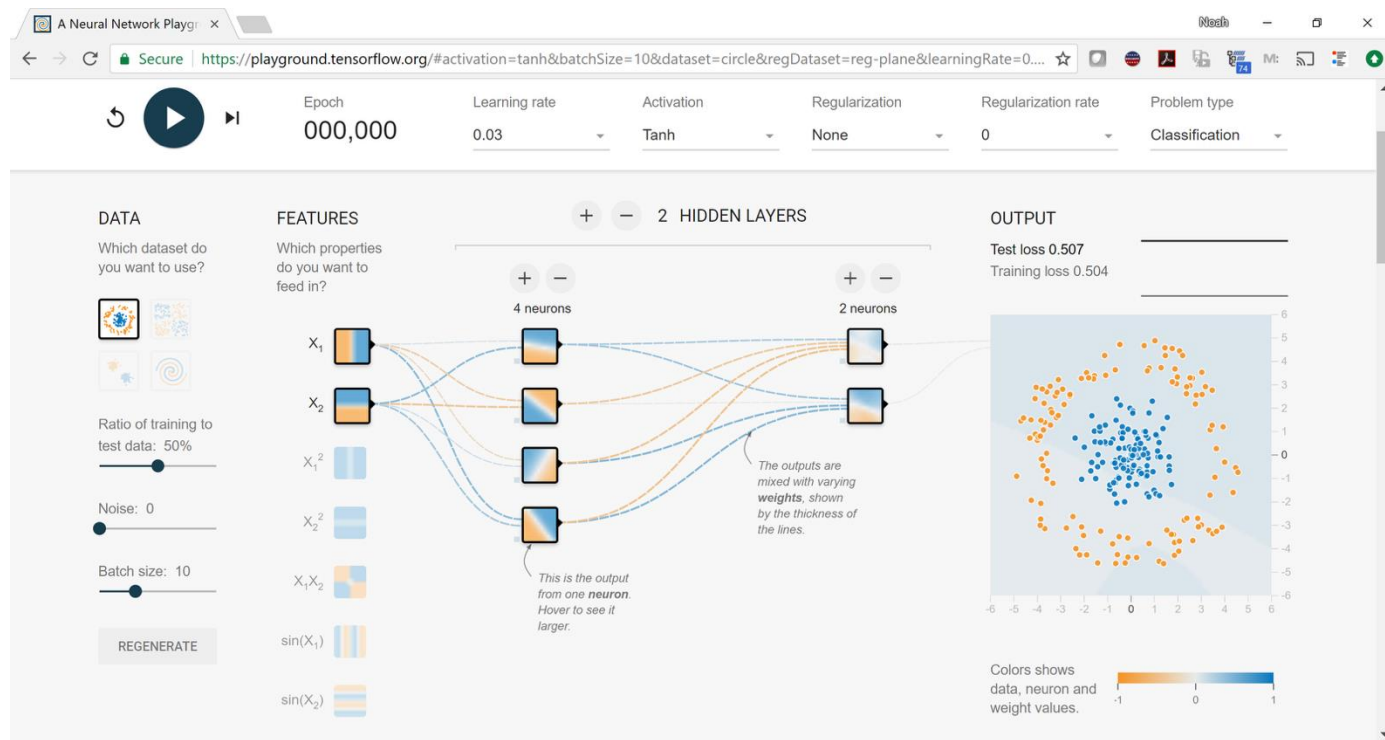
Neural network architecture



```
# forward-pass of a 3-layer neural network:  
f = lambda x: 1.0/(1.0 + np.exp(-x)) # activation function (use sigmoid)  
x = np.random.randn(3, 1) # random input vector of three numbers (3x1)  
h1 = f(np.dot(W1, x) + b1) # calculate first hidden layer activations (4x1)  
h2 = f(np.dot(W2, h1) + b2) # calculate second hidden layer activations (4x1)  
out = np.dot(W3, h2) + b3 # output neuron (1x1)
```

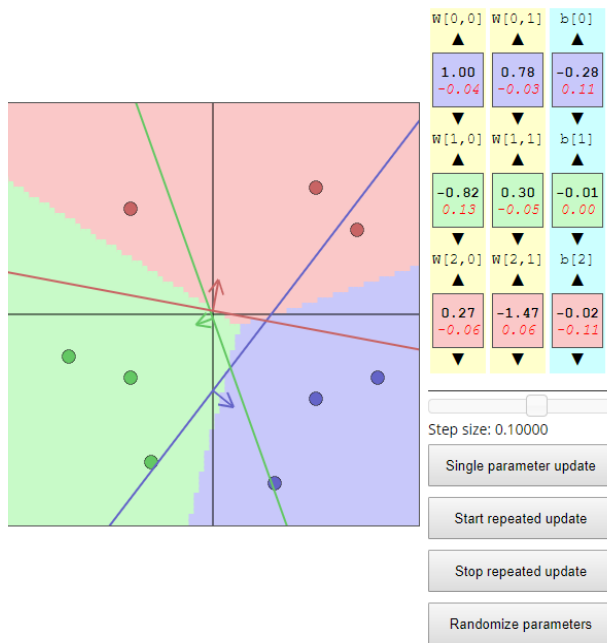
$$f(x) = W_3 \sigma (W_2 \sigma (W_1 x + b_1) + b_2) + b_3$$

Demo: Neural networks



<https://playground.tensorflow.org/>

Demo: Linear Classifier



<http://vision.stanford.edu/teaching/cs231n-demos/linear-classify/>

Questions

1. Why do neural networks need non-linear activation functions?
2. How does an MLP differ from a single-layer perceptron?
3. Compare sigmoid, tanh, and ReLU activations.
4. What's the optimal neural network architecture?
5. How can we optimize/estimate weights/parameters?

Readings

- Neural networks
 - https://d2l.ai/chapter_multilayer-perceptrons/index.html
 - <http://cs231n.github.io/neural-networks-1/>
 - <http://cs231n.github.io/neural-networks-2/>
 - <http://cs231n.github.io/neural-networks-3/>
 - <http://cs231n.github.io/neural-networks-case-study/>

What's next?

- Ga